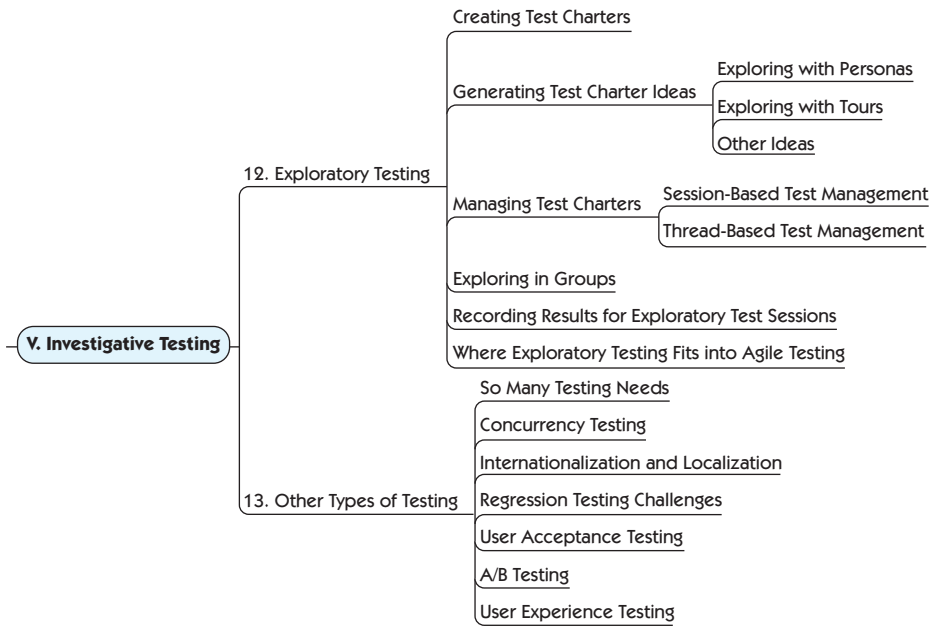
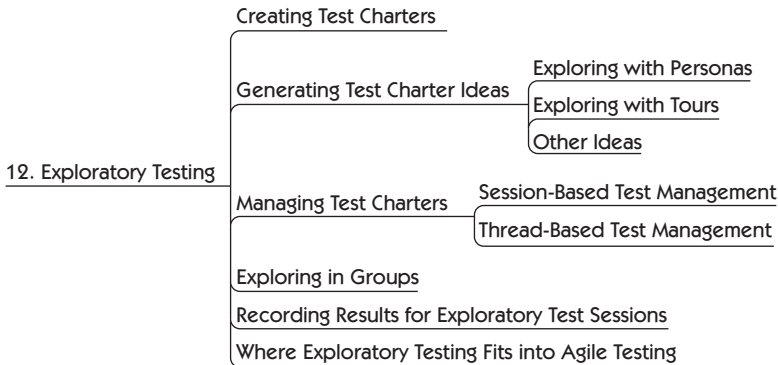




- Chapter 12, “Exploratory Testing”
- Chapter 13, “Other Types of Testing”

EXPLORATORY TESTING



Exploratory testing combines test design with test execution and focuses on learning about the application under test. It requires a mixture of thinking processes: logical, calculating, and conscious, along with fast and instinctive. In Chapter 4, “Thinking Skills for Testing,” we discussed how different styles of thinking are needed for different types of testing. Exploratory testing is one area that particularly benefits from applying different ways of thinking. In this chapter we will share how exploratory testing has developed and how you can practice it in your agile teams.

Exploratory testers must consider many aspects of the product, including its users, how the feature they’re testing relates to the company’s business goals, what ripple effects implementing the feature might have on other parts of the system, and what the competition is doing. In our experience, testers who are skilled at exploratory testing bring enormous value to their teams.

Lisa’s Story

My current team hired its first full-time tester less than two years before I joined as the third tester. Working with testers was new to the company

culture. The team developed everything test-first and did a great job of test automation at all levels. After we three testers had been working together with the team for more than a year, I was surprised and gratified when one of the programmers noted on a company forum that he thought that no amount of automated tests could replace skilled, dedicated testers who know how to do exploratory testing well.

Exploratory testers do not enter into a test session with predefined, expected results. Instead, they compare the behavior of the system against what they might expect, based on experience, heuristics, and perhaps oracles. The difference is subtle, but meaningful.

James Lyndsay explains the subtle difference between scripted testing and exploratory testing in his paper “Why Exploration Has a Place in Any Strategy” (Lyndsay, 2006). He notes:

An automated test won't tell you that the system's slow, unless you tell it to look in advance. It won't tell you that the window leaves a persistent shadow, that every other record in the database has been trashed, that even the false are returning true, unless it knows where to look, and what to look for. Sure, you may notice a problem as you dig through the reams of data you've asked it to gather, but then we're back to exploratory techniques again.

In another paper, “Testing in an Agile Environment” (Lyndsay, 2007), James suggests that one of the roles that testers can play is the “bad customer.” A bad customer goes off the happy path and may even try to break the system. Using your knowledge of potential issues, you can play the part of the “bad customer,” whether that’s someone who’s malicious, in too much of a hurry, or simply incompetent. Here are some examples of actions you might try as this persona:

- Invalid parts of input—characters, values, combinations
- Time changes
- Unusual uses
- Too much—long strings, large numbers, many instances
- Stop halfway/jump in halfway
- Wrong assumptions

- Making lots of mistakes, compounding mistakes
- Using the same information for different entities
- Triggering error messages
- Going too fast



You may not consider yourself to be a skilled exploratory tester, but you may have tried typing in all the special characters on the keyboard or blowing out the buffer of an input field. You can use these instincts in a more mindful way. Elisabeth Hendrickson's "Test Heuristics Cheat Sheet" (Hendrickson, 2011) is a great place to start looking for ideas. Exploratory testing gives you the opportunity to use your ability to critique, assess, and challenge your understanding of the product in a purposeful manner, in order to provide critical information to your stakeholders. Later in this chapter we will suggest ways to give constructive feedback to stakeholders.

Exploratory testing and automation aren't mutually exclusive but rather work in conjunction. Automation handles the day-to-day repetitive regression testing (checking), which enables the exploratory testers to test all the things the team didn't think about before coding. As you explore, you may find additional tests that need more investigation or should be automated. You will likely use automation to set up exploratory scenarios, to monitor log files, or perhaps to explore scenarios that are not accessible through manual means. For complex new features where there are many "unknown unknowns," testers and programmers can explore together as they spike potential solutions to learn enough about the feature to start writing stories and tests to guide development.

There are multiple ways to explore. You may explore alone, or in pairs or groups. In this chapter, we'll discuss a few techniques that have worked well for us, and we've included stories from those who have had other experiences. We encourage you to experiment with these approaches as well, using lightweight strategies to manage them. The bibliography for Part V, "Investigative Testing," contains many books, articles, and other resources for learning more about this powerful testing approach. Make exploratory testing an important part of your toolkit when testing stories, features, and the system as a whole.

CREATING TEST CHARTERS

A test charter outlines a goal or mission for your exploratory session. There is no one right way to create a test charter, but we'll look at a few different methods. As always, it's best to experiment and find the one that works for you. You may find that one style works best for session-based test management (SBTM), while another works better for testing with thread-based test management (TBTM). Charters may range from happy path functional validation (although this is perhaps less valuable) to exploring failure modes, and performance and scalability.

Elisabeth Hendrickson's *Explore It!* (Hendrickson, 2013) has a section on creating good charters. It takes practice to find the right level of detail for your purposes. Too specific means that you don't have enough room to wander off the beaten path to make unexpected discoveries. Too vague or broad doesn't give enough focus and may cause you to waste time.

Experiment with how you word charters, as well as with the number of charters you create. Elisabeth notes that "a good charter is a prompt: it suggests sources of inspiration without dictating precise actions or outcomes" (Hendrickson, 2013, p. 16). One template Elisabeth has found that works well and gives enough guidance and focus is

```
Explore . . . <target>  
With . . . <resources>  
To discover . . . <information>
```

It's better to have multiple charters, each of which is concise and focuses on a single area, for example:

```
Explore editing profiles  
With real usernames  
To discover if there are instances where username constraints are not  
enforced
```

Another way to create a test charter is a mission statement and areas to be tested, for example:

Analyze the edit menu functionality of Product X

And report on areas of potential risk in Operating System Y

The simpler you keep a charter, the easier it is to stick to it. However, James Lyndsay reminds us that the broader you make a charter, the easier it is to consider distractions within the charter (Lyndsay, 2014). The breadth and specificity of the charter are tools to guide exploration. For example, a charter that says “Performance Test X” really gives no guidance.

Let’s use an example of a web-based toy store to write a charter for the end game during a delivery cycle. *Note:* The feature would have been explored for the workflow as soon as it was completed.

Explore shopping for a new toy

With a real live user

To identify potential bottlenecks and unexpected disadvantages

Again, we’re not suggesting that one way is better than another. On agile teams, we move at a fast pace, so we want to keep focused on the stories and features we’re currently developing. At the same time, we have to keep the big picture in mind and make sure new code doesn’t cause unintended effects elsewhere in the application.

About Sessions and Charters

Matt Heusser, who writes and consults about testing and is based in the United States, shares his experiences with charters and session-based testing.

A couple of years ago I was consulting for a large corporation that used an agile method. The business domain was incredibly complex, with legacy systems composed of many layers added by different people over decades. Each team contained everything needed to ship software, including support and operations, and each team

determined its own work process independently. To try to share information, the company had a monthly “tester community of practice” meeting.

The “charters” the team adopted were one to three pages long in MS Word and were relatively prescriptive. It was an improvement from what they did before, but not what I think about when I say “test charter.”

For me, a charter is a mission for testing. In an agile environment, where we assume regular functional testing as part of story work, a charter might address a risk we want to investigate or invest time to mitigate. Here are some examples:

- Test <feature> under <new browser version>
- Test <system subset> with tablets
- Test for failures with <third-party web services that are usually <up>
- Test multi-user and race conditions with <feature>
- Additional exploratory penetration testing for the new web front end as a system
- Customers see <X> but we are unable to reproduce the problem; can you troubleshoot?

Let’s take another look at these charters. The examples are the intersection of a specific risk and a feature, but they don’t have to be. I like my charters to be less than 200 characters long—most of the time, they fit into a tweet. That tweet is the title. If you have specific concerns or spend five minutes brainstorming, you might create a list of test ideas that become the “body” of the charter.

You can use charters in many ways. I’ve made heavy use of charters to help manage the release process with limited time. I tend to think of charters as small chunks of work, usually 30 to 60 minutes. Of course, if we identify a risk that can be investigated during the iteration, doing it earlier is better. When the charter makes sense to tie into a story, we can just put it in the story notes. When the charter addresses a crosscutting concern or is unrelated to current stories, I might suggest creating a new story.

By taking notes and debriefing a fellow team member on what we’ve done after the session, we have a chance to increase team knowledge about the system and test practices. This serves as a sort of mini retrospective for testing.

Start with the style of charter that seems most workable for your team. Trying different formats is a nice way to shake things up and help you see your software with a fresh perspective. You can think about the templates we've provided as training wheels until you find the one that works for you.

GENERATING TEST CHARTER IDEAS

There are a few techniques that can help generate test ideas that we'll share in this next section. It is not an exhaustive list, but we hope it will trigger some of your own ideas.

Exploring with Personas

Personas are a way of understanding your end users, and many companies create personas as part of their marketing strategy.

Jeff Patton (Patton, 2010) and David Hussman (Hussman, 2011) are among the many practitioners who create pragmatic personas to identify who actually uses a product. Personas are a good way to imagine different ways people will use an application. We explained how we use personas for usability testing in *Agile Testing* (pp. 202–4), but we think the concept can be taken beyond usability. If you currently don't have defined personas, conduct a quick workshop with your team to discover at least some of them to give you a start. We've mentioned James Lyndsay's "bad customer," and we've shared two personas (see Figures 12-1 and 12-2) that Mike Talks has used for testing login account functionality and security.

Concentrate as a user on the following chunks of functionality:

- Suspend your account.
- Deactivate the RSA token linked with an account.
- Delete your account.

Potentially this user will need help from the help desk.

The Security Worried User: This user has concerns about using the login service and wants to be protected from anything bad happening. After all, you hear such terrible things in the news about people's identities being stolen.



Figure 12-1 Security Worried User persona

Try to support the Help Desk admin with the following:

- Search user.
- Authenticate identity.
- Deactivate RSA token.
- Delete account.

Flows to follow:

- Create new account.
- Set account to be Help Desk User or Help Desk Admin.
- Delete account.
- Get reports of activity.

By its nature you will also be touching upon

- Monitoring activity
- Validation

Personas are a great way to look at your application from different angles.

The Help Desk Admin: The administrator can give permissions to other users to make them Help Desk Users or Help Desk Admins. The Help Desk Admin goes beyond the Help Desk User in what the person can support and change in people's accounts.



Figure 12-2 Help Desk Admin persona

Lisa's Story

My team experimented with creating a separate project for system testing a complete rewrite of our agile project-tracking product. A tester, a designer, and a marketing expert teamed up to identify various personas representing our users. Developer Denise and Product Paul were two representatives of our product's users. We wrote charters as user stories so we could track them in our online tracking tool. This was a good way to track the testing we felt was needed, but not a good way to capture the results of our exploratory testing sessions. See the section "Recording Results for Exploratory Test Sessions" later in this chapter on how we did that.

Persona: Paul is the project manager for Agile Toys. In his weekly iteration planning meetings (IPMs) with the team, he goes through stories in the upcoming iteration, answers questions, updates the stories with point estimates, and rearranges stories in the backlog according to priority. He typically uses his iPad in the IPM to make the changes. At his desk, he uses Chrome on a MacBook Air. Paul's main usage of the tracking tool is keeping the backlog organized and prioritized.

Charter: Explore as Paul, the project manager, in an IPM to discover any issues with concurrent updates to stories from different devices.

Scenarios:

- Pre-IPM backlog prioritizing
- Iteration planning meeting—updating stories

Each of us sets aside an hour a day for exploratory system testing for what we call “mini group hugs,” where each tester chooses a persona and a browser and tests concurrent usage of the system. We recorded our test results on our team wiki.

Using this process, we’ve found important bugs that weren’t found while doing exploratory testing on the individual user stories. For example, an exploratory testing session with a charter of updating stories as Paul would during an IPM turned up several new bugs related to concurrent usage.

If you use personas, make sure your whole team understands them and how they can help with testing. Make them visible, perhaps by pinning their pictures and profiles on the wall. It is a good way for the programmers to get a better understanding of the users, and it also helps raise awareness of the value of exploratory testing.

Exploring with Tours

Tours can be another useful tool to generate ideas for exploratory charters or to get familiar with a new product or capability. This technique uses a metaphor of tourism and can add useful variation to your explorations, uncovering issues you may not see otherwise. James Whittaker has described some unique exploratory testing tours (Whittaker, 2012).

For example, as a tourist, perhaps you want a strategy for seeing the most important sights in London. Who you are, or what your goal is, will determine your strategy. Whittaker suggests that visiting students would approach this situation much differently from a group of flight attendants who are there for a weekend. A similar approach can help you explore your software features. Check out Whittaker’s defined tours, such as the Guidebook Tour (looking for bargains, shortcuts) or the Landmark Tour (hopping through an application’s landmarks). In the

Landmark Tour, you would identify a set of software capabilities (the landmarks) and then visit those landmarks, perhaps randomizing the order. Changing the sequence of events may cause an unexpected error to occur. These are good places to start, but make them your own. For example, try combining personas with a tour to visit your most important landmarks.

Tours can be done at any time, but Janet has had great success defining tours for the end game, when you think your new release is ready to ship. Often this will give your team extra confidence that the most important features in your product work as expected. Be creative in how you document these touring sessions. Perhaps you can create a visual map to show where you've been. See the bibliography for Part V for links to explore some of the possibilities.

Markus Gärtner (Gärtner, 2014) recommends debriefing after completing each tour. As you debrief, you'll identify more charters, which will take you deeper into areas you briefly touched on in the tours.

Other Ideas

Some teams base their charters on their story acceptance criteria. For example, you may want to explore error handling, perceived response time, and complementary features.

If you have identified risks during story elaboration, you may create risk-based charters that will highlight likely problem areas or areas of uncertainty. A conversation with your programmer is an excellent source for identifying architectural risks and makes a great driver for test charters.

David Hussman (Hussman, 2013) suggests creating journeys to take the personas you've identified someplace interesting. Once you've learned more about the personas through story mapping, imagine where they might like to go. These journeys might also be a useful way to explore your system after it is built. For example, if we refer back to the story

map in Chapter 9, “Are We Building the Right Thing?” (refer to Figure 9-2), one way to test it might be to describe a charter as a possible journey:

Journey: Search by keyword, select an email, add sender to contacts, and reply.

Charter:

Explore journey

With different folders, different senders

To discover if flow hangs together

In Chapter 13, “Other Types of Testing,” we will look at some ways to use exploratory testing for several quality attributes beyond the scope of functional tests.

MANAGING TEST CHARTERS

By now you should have some great ideas for creating your exploratory test charters. The question now is, How do you keep them straight? There are a few different ways, and we’ve shared a couple of stories from exploratory testing practitioners about how they manage their charters.

Session-Based Test Management

Session-based test management (SBTM) is based on the idea of creating test charters or missions for a testing idea, exploring uninterrupted for a specific time period, recording the results, and following up with a debriefing session. We mentioned this technique in *Agile Testing* (p. 243) but will share a few other ideas we’ve gathered. For example, Bernice Niel Ruhland told us she uses SBTM to help train testers. The debrief session is a great way for her to provide immediate feedback to the new testers.

Managing Regression Tests with SBTM Charters

Matt Heusser shares his experiences with SBTM and how he uses it to manage a manual regression suite.

SBTM fits well in an agile development process. It radically improves the performance of the test process so that it can fit into tight time boxes in a way that stands up to scrutiny.

SBTM uses a cognitive, thinking, and investigative approach, which also makes it work well in an agile setting. Using SBTM, I've managed to compress regression testing to fit within an iteration for teams that have almost no test tooling and no customer-facing automated testing infrastructure. We do use automation to help, for example, when we generate test data and make build environments faster.

To make it work, the software has to be reasonably high quality before you get to the "final shakedown" test. Otherwise, the find/fix/retest loop kills you. Jon Bach's SBTM paper (Bach, 2000) gives more specifics, but here's one way to adapt the method in an agile context. Consider this exercise for a whole agile team:

1. Make a list of risks for the product in charters, 30 to 60 minutes apiece.
2. Put all the lists on a wall, whiteboard, or projected page.
3. Sort the list by risk. Dot vote if you'd like to determine priority.
4. Put the list on a kanban board.
5. Team members pull one charter at a time and test, reporting and fixing bugs as a whole team as they go.
6. At any time, anyone can change the order of the cards or add new cards.
7. When you run out of time, stop testing. Look at the known issues and the risks not tested, and ask yourself, "Are we ready to ship?" Conduct a Roman vote, thumbs up or down.

SBTM is a whole-team, people-centered approach to testing. It can help with designing a test strategy for a legacy application. If your application has complicated rendering code or business logic in the graphical user interface (GUI), consider using SBTM in combination with high-level GUI smoke tests, rather than trying to cover everything with automated GUI tests. This combination can provide better coverage at a lower cost and let your team make better use of their test time.

Also consider SBTM when you see risks in platform or third-party compatibility. For example, SBTM can help you catch failures, resulting in an upgrade to jQuery or a third-party open-source framework.

SBTM is useful if you need to support a new platform. For example, if your team is unexpectedly faced with supporting mobile devices, the touchscreen platform poses usability and compatibility risks. You can manage these well through session-based testing.

Complex business domains, high cost of failure, and other scenarios where early bug detection is critical are all motivating factors to use SBTM. By all means, tweak the process to improve the quality of the code before it gets to final test—but you can also add a layer of SBTM as a compensation mechanism. When the consequences of a mistake are high, you can reduce risk with SBTM.

Exploratory testing provides a safety net. Managing exploratory testing through sessions provides one more layer of protection and includes a feedback mechanism to help the team learn to improve in its exploration.

Try SBTM with your team. As with any testing technique we mention, there's no one perfectly correct way to do it. Conduct a session, see how it works for you, inspect, and adapt. Check the Part V bibliography for resources to learn more, including James Lyndsay's "Adventures in Session-Based Testing" (Lyndsay, 2003).

Thread-Based Test Management

Thread-based test management (TBTM) is less rigid in terms of time-boxing a session than SBTM. It works on the idea of organizing tests around threads of activity, rather than test sessions or artifacts. A thread does not imply a commitment of a specific time box as SBTM does. Its flexibility may lead the tester in different directions. TBTM may work better in some situations with rapidly changing priorities or frequent interruptions.

Converting from Session-Based to Thread-Based

Christin Wiedemann, a tester, trainer, and speaker from Vancouver, Canada, shares the challenges she faces with SBTM and how she adapted the process to TBTM with mind maps.

Over the years, I have repeatedly found myself in environments that are too volatile for more rigid test approaches to work well, especially as deadlines draw nearer. One example is a small team of four developers and two testers, working on a product for an external client. We had adopted a hybrid test approach, mixing scripted manual test cases and exploratory testing, organized in sessions structured according to the principles of SBTM.

As we got closer to the launch date, the pace increased drastically. As a result of increased pace and the rapid changes, we abandoned the scripted tests and ran only exploratory test sessions. However, we would typically start a test activity but quickly get interrupted and not complete the task. Working in this manner caused a great amount of stress, since it felt as if we never finished anything and never made any progress. We also came to realize that in some cases we felt hemmed in by the actual session. Even if the conditions changed or something urgent came up, we still felt obliged to finish the session before we started a new activity. Something had to change, so we decided to give TBTM a try.

The first step was to make a mind map containing all test ideas as threads (see Figure 12-3). We grouped the test threads in different ways. Most groups represented function areas (e.g., installation), but other groups were formed based on the type of testing (e.g., stress testing). After a while we realized that we wanted a third type of group that corresponded to issues we encountered frequently—potential problem patterns or heuristics such as log files. Each group corresponded to a node in the mind map. In some cases, short notes were needed to explain the thread, and we simply added those notes to the mind map. We also prioritized the function areas based on risk and added this information to the mind map by labeling the different nodes with numbers. This simple, yet powerful, test plan took us less than an hour to draft.

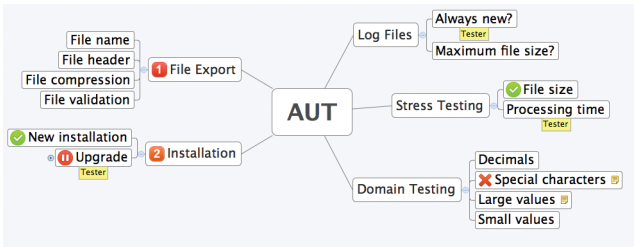


Figure 12-3 Mind map test plan

During the test period, we were constantly updating the mind map, and it always gave an accurate picture of the current status of testing (see Figure 12-4). As soon as someone started working on a thread, we would make a note in the mind map. Threads grew by the addition of short notes summarizing our testing and our observations, and we added new threads as we came up with new ideas. When we found defects, the corresponding thread was marked with a red cross, and the identification number from our defect-tracking system was added together with a short description (see Figure 12-5). Threads for which we felt sufficient testing for delivery had been done were checked off in green.

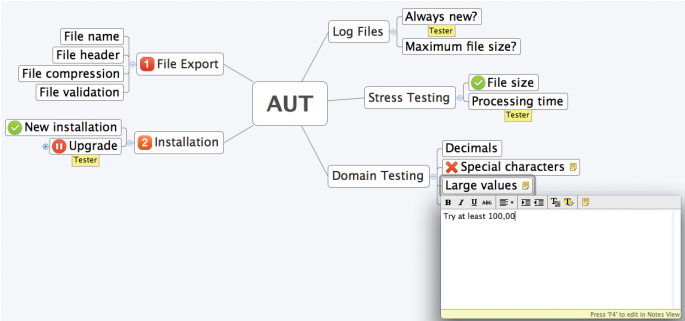


Figure 12-4 Test results with notes in a mind map

Initially our goal was to also write daily status reports containing a few short notes on what had been tested, but in reality we kept this up for only four days. With constant updates to the mind map, we really

didn't feel a need for additional reporting. At the end of our test period, we used the latest version of the mind map as our test report, which was sent to the customer.

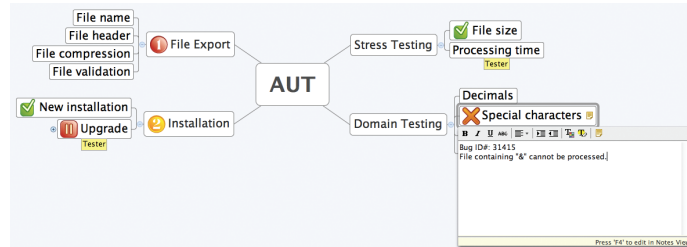


Figure 12-5 Defect logged in a mind map

What was the main difference between this project and previous projects I had worked on? The first thing that comes to mind is that this time we actually used the test plan to guide our work! In addition, the test report was continually read and reread, both to review our test results and to plan the testing on a subsequent project. Keeping the mind map up-to-date was much easier than updating any other kind of status document, which meant that it actually got updated. As it turned out, even the developers liked it—they preferred looking at the mind map over using our bug-tracking tool!

TBTM works very well under some circumstances, but I've found that mind maps are great tools regardless of the test approach. Nowadays I always use mind maps to plan testing. This not only allows me a higher degree of control without causing additional overhead, but it also makes it easier to initiate discussions and obtain feedback within the project team.

Christin's team grouped threads by functionality or type of testing, but threads can also be based around a feature or a story. They can be organized based on common resources—for example, small-scale functional data threads versus large-scale performance threads, or threads that focus on failure modes versus threads that focus on happy path workflow.

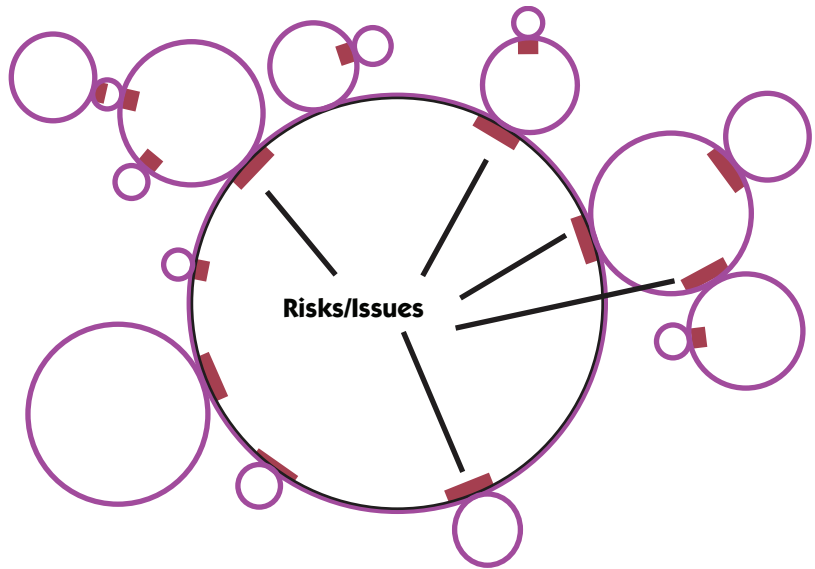


Figure 12-6 Fractal representation of TBTM



Adam Knight (Knight, 2011 and 2013) tackles his organization's large-scale data testing using thread-based testing, which allows testers to work on threads in parallel defined by test charters. Figure 12-6 is a representation of how he explains the benefits of an exploratory testing approach using threads to new testers in his company. They start with a feature area, idea, or risk in the center. As a flaw is discovered, they expand a new set of tests on that discovery. For most discoveries, this is done under the scope of the thread. If discoveries are made that are too large to be considered within the scope of the thread, a new thread is created to explore that area.

That mini exploration will result in a more targeted testing exploration around that feature area and can be represented as a circle off the original. In this way, the testing effort within each thread naturally focuses on the problem and risk areas as they are discovered.

Markus Gärtner (Gärtner, 2011) uses a slightly different tactic, which he calls Pomodoro Testing. He uses shorter sessions of 25 minutes and continues developing his testing mind map during the debriefings. You

can find links to more about Adam Knight's experiences using TBTM and Markus's Pomodoro Testing in the bibliography for Part V.

EXPLORING IN GROUPS

Generally, teams think about exploring as an individual activity, or maybe an activity done by a pair. However, group exploring provides unique opportunities to discover issues or missing features in a new product or major update. We have facilitated exercises in shared exploration, and the results demonstrate the same thing—diversity creates different ideas.

Bernice Niel Ruhland (Ruhland, 2013b) uses this approach once in a while for more complex, riskier areas of a product, especially when time is working against the team. She had the testing team, programmers, and business analysts (BAs) participate in the testing. She recounts:

We used an Excel spreadsheet to define the tests as we had specific test paths based upon coding risks to explore. In some cases critical bugs were fixed before we even finished the testing session. I received positive feedback from this approach. And of course how much or little documentation we provided changed based upon what we were testing and the testers' experience level with the functionality.

Lisa's Story

Spread the Testing Love: Group Hugs!

When our team is preparing a major new release, we sometimes organize "group hugs," where the whole team, or a subset of it, joins in for testing. Sometimes it's only testing team members, sometimes the entire product team, or something in between, but it's always useful. Some people refer to this type of activity as a "bug hunt," but we feel it's a positive activity that demonstrates our passion for building quality into our product; our focus isn't about finding bugs, but about consistency and confidence.

The iOS Group Hug

Recently we released some new features to our iOS app. We asked the team for volunteers for a group hug. Programmers, testers, and marketing folks joined in. We used a shared Google doc for the session, where we noted which device and version each participant was using, as well as already-known problems.

Our team is geographically distributed, so we used a videoconference meeting to communicate during the testing. People in each of our office locations gathered in one room, and remote people joined individually. We find it's helpful to be able to physically talk to each other. For example, if more than one person finds the same bug, we can avoid duplication in reporting it. Also, talking through what we've tried gives other team members ideas for interesting tests.

During the group hug, we found some new bugs, as well as some usability issues. Generally, the feedback was positive, and we were able to release within a few days.

Involving multiple team members in one testing session is expensive. However, we've needed to do it only for major, risky new features where concurrency is critical, and generally one group hug is enough to provide necessary feedback.

The Place for Group Hugs

We build quality into our product with test-driven development (TDD), acceptance-test-driven development (ATDD), and constant pairing and collaboration. We have multiple suites of automated regression tests providing continual feedback in our continuous integration system. We do have automated tests for concurrent changes, but they don't cover every possibility. In addition to the extensive exploratory testing we do on each new feature, the group hugs provide a quick way to get information that we can't get in our normal process.

Consider group exploring (see Figure 12-7) if concurrency is a vital feature of your product, and your automated tests and exploratory testing by individual or paired testers can't cover every scenario. Use these sessions judiciously, and add only as much structure as you need. Sharing a document where everyone records the results is helpful, but in some cases it's too heavyweight. The same goes for preparing charters in advance.

Another option is exploring with a trio (programmer, tester, BA) aimed for fast feedback. Julian Harty calls this "Trinity Testing" (Harty, 2010). The programmer can give feedback on the charters or testing paths based on his or her knowledge of the code, the BA on the business risk. When issues arise from testing, the BA can explain any business impact while the programmer assesses coding risks and time to fix the problem.



Figure 12-7 Exploring in groups

However you choose to run your group session, be prepared to learn from each one, so you can improve your next group testing session and learn even more about the software you're testing.

RECORDING RESULTS FOR EXPLORATORY TEST SESSIONS

It is important to record your exploratory test results and coverage as well as to share your thoughts with others. Some of the reasons to record results or make notes are: it gives you an opportunity to review your results with a peer and have a meaningful conversation about your findings; it allows you to track progress and issues as Christin showed with her TBTM story; it provides the opportunity to review your own results for later testing or if you want to review charters; and last (and our least favorite reason), it demonstrates to management what you've done if problems occur.

Recording can be as simple as taking notes on paper, on a wiki page, as a mind map, in a text document, or on a session sheet. Record issues,

unexpected behavior, or features that seemed to be lacking. Hold a debriefing session to go over what you discovered. You're likely to think of ideas for future sessions as you discuss your most recent one with peers.

Keep your notes brief and simple. If multiple team members are exploring individually or as a group, agree on one format for note taking so you can easily understand each other's findings. Post results in a place where your whole team can see them for reference, and use these notes to improve your testing.

Lisa's Story

When we wrote charters based on personas, we created a chore (a task) in our team's online tracking tool for each charter. This was a good way to track the testing we felt was needed, and we could easily assign ourselves charters and see at a glance who was working on which charter. We could also track which charters were complete, in progress, or not yet started. We tried putting the results of our sessions in these chores, but we didn't find it convenient to refer back to the results this way. Next, we experimented with writing up our test session results on our wiki. Here's an example:

Explore being Paul, the project manager, in an iteration planning meeting.

Browsers: Chrome and Firefox

Initial setup: On Test01, used project 101 populated by the test data fixture.

Observations:

- Deleting stories required a reload.
 - The velocity shown for iterations in the backlog didn't change as I reprioritized stories until I reloaded.
 - Need to do more exploring in projects with custom point scales.
-

Spreadsheets are a simple way to record exploratory test sessions. Adam Knight uses Excel for the top-level charters and exploratory note taking because it provides flexibility to present test results in many forms: text,

graphs, tables, and external links. He then has a set of small macros that provide input boxes to allow very fast input of test notes and tracking of status. He can record status but also can indicate if an issue is “off the charter” and merits the creation of another charter to follow up.

Bernice Niel Ruhland stores her threads in one Excel tab with additional columns for testing notes. It’s like reading a story about the testing. At a glance she knows how many threads are done, in progress, and not started. It also allows her to review any issues quickly.

There are products available that record notes as you do exploratory testing. See the “Tools” section of the bibliography for a link to one of these. Some session-recording tools let you record your keystrokes and the pages you’ve visited so that you can go back and reproduce your steps if you find something worth investigating further. They even let you specify how many pages you want to keep in memory or what type you want to save.

Janet’s Story

I decided to try a tool I heard about at a conference called qTrace from QASymphony. (Note: I am not recommending this one over any other.) It is a recording tool for exploratory testing and captures screen shots, notes, and other details. I created a charter for exploring my own website, www.janetgregory.ca:

Explore the blog page

With the search function

To discover if it returns what is expected

I set a time limit of 30 minutes, but with the focus narrowed, I immediately found two issues that I had not noticed when I first checked the search function. The screen prints allowed me to check where I had been without thinking twice. The interesting thing was that I found one issue that bothered me enough that I put in a new request (story)—to search only in blog posts rather than the whole site—and one defect—showing posts authored by Mark (strange, since my name isn’t Mark, and nobody else posts) that weren’t posts, as highlighted in Figure 12-8.

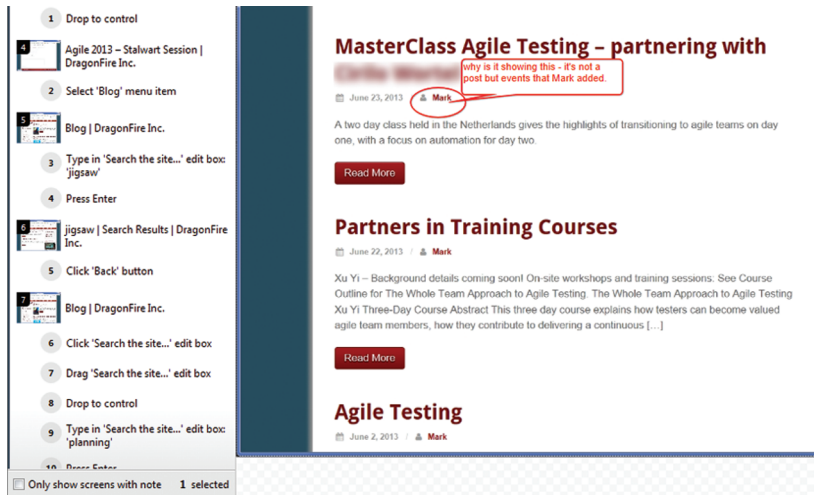


Figure 12-8 Exploratory test session using qTrace

Some recording tools record extensive additional information, such as data about the environment. This helps when you need a reproducible bug so that programmers can write a failing automated test and a fix. You may also use this information to write new user stories and new tests for features you found lacking or to tweak existing features you'd like to improve.

Whatever method you choose to record your results, remember to make them accessible to the whole team. Share the valuable knowledge you gained by exploratory testing.

WHERE EXPLORATORY TESTING FITS INTO AGILE TESTING

We think that exploratory testing is an integral part of agile testing, and it's important to see where it fits in an agile context.

Part IV, “Testing Business Value,” was all about testing for business value. Another way to think of it is testing ideas and assumptions early—before we start coding. That type of testing is about building the right thing. There is another kind of testing designed to answer the question,

“Are we building it right?” We included exploratory testing in Quadrant 3 of the agile testing quadrants because we are usually exploring the workflow to see if we delivered the business value we anticipated. We challenge our assumptions to see what we did not think about earlier when building.

Consider our levels of precision from Chapter 7, “Levels of Precision for Planning”—the product release, the feature, the story, and the task—and what type of exploratory testing might be useful at each level.

- **Product release level:** This is where you would test an integrated product delivery to the integration team, or a release candidate during the end game. This would be a good time to explore dependencies between teams and high-risk workflows and perhaps do tours.
- **Features:** Once all associated stories are “done,” you can explore the complete feature. At this level, good candidates for exploration are feature workflow and interaction with other applications. This might be a good place to try what Lisa calls “group hugs”—more than one person exploring on a charter.
- **Stories:** Once the story meets the expected results—initial coding has been completed and all the automated tests specified before or during coding pass—you can start exploring. Think about the development risks, boundary conditions, more detailed functionality issues, and different variations of formats or states.
- **Tasks:** Exploring at this level would happen during coding. Examples might be programmer exploration on an API, pairing on performance issues with a tester, or maybe even exploring some of the boundary conditions on strings. Consider programmer-tester pairing to create exploratory charters for the code being developed.

Agile teams benefit from continual exploring. When you’re brainstorming new features, you can explore how your different personas might use them. You can explore released software to identify what features may still be missing. Focus your exploratory testing by using charters, tours, heuristics, or session- or thread-based test management. What you learn will help your team improve quality, not only in the impending release,

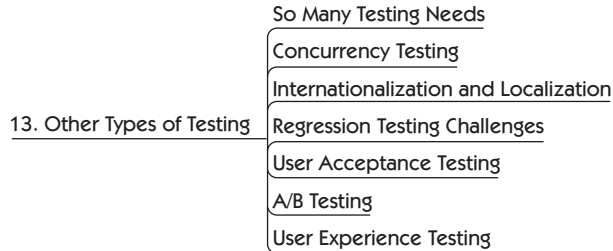
but as the product evolves in the future. Use the resources in the bibliography for Part V to grow your team's exploratory testing skills.

SUMMARY

Whichever techniques you use, exploratory testing is a powerful way to test new stories and features as they are incrementally delivered.

- Use exploratory testing approaches
 - In conjunction with your automation strategy
 - To give feedback quickly to stakeholders to see if you are building the right thing
 - To prepare for releases with manual regression testing
- Experiment with different methods to create your test charters.
- Experiment with new test charter ideas, such as using personas, tours, or journeys.
- Manage your test charters with SBTM or TBTM or your own approach that meet your needs.
- Practice to build your exploratory testing capabilities.
- Exploratory testing adds value in most, if not all, agile development activities.

OTHER TYPES OF TESTING



In Chapter 8, “Using Models to Help Plan,” we reintroduced the agile testing quadrants. In the section on testing early, we covered Quadrant 1 and 2 tests commonly used by agile teams. In Chapter 12, “Exploratory Testing,” we outlined several approaches to exploratory testing. There’s a seemingly endless list of different types of tests that may be appropriate for your application. For example, some domains require unique approaches, such as the specialized testing techniques needed for business intelligence and data warehousing software, which we will cover along with contexts in Part VII, “What Is Your Context?” In this chapter, we’ll cover a few different types of testing other than functional testing and focus on ones we don’t normally talk about in agile teams but that we think are important.

Remember that the numbering of the Quadrants doesn’t indicate when different types of testing should be done. Q4 tests should be considered as soon as you start discussing a new feature or theme. For example, if there is a constraint that all pages in the application must respond in less than two seconds, make sure everyone on the team is aware of it. Capture that requirement as tests, and keep it in mind during development.

In *Discover to Deliver* (Gottesdiener and Gorman, 2012), Ellen Gottesdiener and Mary Gorman represent the quality attribute dimension in two different areas: operations and development. In Chapter 11 of *Agile*

Testing, we covered some of the basic operational attributes that need testing, such as security, reliability, availability, interoperability, and performance. That is not a complete list, but it gives you an idea of the attributes and constraints we need to consider. The Quadrants can help you brainstorm about the different kinds of testing you need to do on your particular feature or product.

SO MANY TESTING NEEDS

Our first Extreme Programming (XP)/agile teams back in the late 1990s and early 2000s were focused on finding out what our customers wanted and then delivering that functionality. We and our teammates tended to be generalizing generalists, and we sometimes neglected areas such as security, reliability, accessibility, performance, and internationalization—at our own peril. It became clear that agile teams must be diligent about all aspects of software quality, even if our business stakeholders don’t mention them.

New types of testing have accompanied the proliferation of platforms, devices, and technology. Many consumer products have software that must be tested. Fitness machines have embedded software. The Tesla automobile has an API! At the same time, outside forces require more test coverage. Security threats to software systems increase all the time, so we must find better ways to test security. As hardware advances in processing speed and graphical display quality, we have to ensure that our products perform well and look good.

Your team may need additional infrastructure to accommodate some types of testing. Let’s say you have a reliability requirement of no more than one failure in 1,000 transactions, or the need to do soak testing. Your team may want a second test environment to be able to drop stable code for reliability testing. You may need extra monitoring tools for watching overnight performance against minimum requirements. If you’re testing an Android app, you’ll need lots of devices in your test lab, and possibly some outside help.

We’ve heard of lists of more than 100 kinds of testing (see the bibliography in Part V, “Investigative Testing,” for an example), although we find

that those lists often contain some duplication. They include the usual ones you would expect to see, such as load testing or browser compatibility testing. However, they also tend to list “types” such as “agile testing,” which really isn’t a type but an approach. When you start thinking about all the different kinds of testing, you might start to wonder when you’ll possibly have the skills, much less the time, to do it all, and it can be overwhelming to new testers. One of the problems we see is the vocabulary misalignment with so many different types. For example, when new people come into the organization and talk about integration tests, everyone else assumes they share the same definition, when perhaps to them, integration testing has a totally different meaning.



We suggest keeping your list of test types as simple as possible and creating a common understanding within your organization. You might start with the types that Wikipedia lists (Wikipedia, 2014l) and differentiate among types, levels, and methods. Bernice Niel Ruhland (Ruhland, 2014) maintains a list of testing types specific to her team, which includes an explanation of how her teams do or do not use a technique. She finds keeping a list helpful for onboarding new testers, and it provides a standard vocabulary shared by everyone in the department. Here’s one example from Bernice’s list of test types:

End-to-end testing is used to test whether the flow of an application or product is performing as designed from start to finish. It typically represents real-world use cases by walking through the steps similar to the end user. The purpose of performing end-to-end tests is to identify system dependencies and to ensure that the correct information is passed between database tables and the application’s modules and feature. We typically perform end-to-end testing for custom-development projects.

Let’s look at a few different types of testing that people often neglect or struggle with on their agile teams. This is not meant to be an all-encompassing list. The goal is to make sure that we remember to do certain important types of testing that often get overlooked in the rush to deliver new features frequently. In Chapter 18, “Agile Testing in the Enterprise,” we will look at how some of these crosscutting concerns affect development organizations with multiple teams.